Learning What to Learn: Hippocampal Memory Consolidation for Continual Model Adaptation

Anonymous Author(s)
Affiliation
Address
`email`

Abstract

Contemporary language models are trained on internet-scale corpora and scaled to massive parameter regimes, incurring substantial costs in training, deployment, and continual adaptation. Human cognition, by contrast, is selective and experience-driven, relying on hippocampal mechanisms that preferentially encode salient, novel, and behaviorally relevant experiences and coordinate their gradual consolidation into long-term neocortical memory. We propose **Hippocampus-Augmented Transformer (HAT)**, a memory-centric continual learning mechanism inspired by hippocampal–neocortical consolidation in the mammalian brain. HAT enables language models to learn *what to learn* from interaction by coupling online experience acquisition with offline memory consolidation. During interaction, a base model (the *Cortex*) processes inputs while a *Hippocampus Agent* encodes interactions and feedback into structured memory traces and selectively retains informative experiences based on uncertainty, novelty, and supervision signals, optionally querying an external oracle when needed. Selected experiences are progressively consolidated into a persistent *Neocortex* memory and subsequently replayed during slow-wave sleep (SWS), where the Cortex is updated through parameter-efficient fine-tuning. This wake–sleep separation transforms sparse, interaction-driven feedback into reusable training signals, enabling continual self-improvement without indiscriminate data accumulation. We evaluate HAT on knowledge-intensive question answering, instruction following, and personalization benchmarks, demonstrating consistent improvements over fine-tuning and retrieval-augmented baselines under continual adaptation settings.

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a broad spectrum of natural language tasks [Brown et al., 2020, Touvron et al., 2023, OpenAI, 2023]. Yet a fundamental tension remains: once training concludes, a model's knowledge is *frozen*, and adapting it to new domains, user preferences, or corrected information requires costly retraining on carefully curated data. This limitation is particularly acute for lightweight, on-device language models in the 0.5–10 B parameter range, where users interact with a personal assistant that must evolve over time but lacks the infrastructure for large-scale retraining.

Existing approaches to post-deployment adaptation fall broadly into two categories. *Retrieval-augmented generation* (RAG) methods [Lewis et al., 2020, Guu et al., 2020] augment the model's context with external knowledge at inference time, but they do not update the model's parameters and thus cannot internalise corrections or new skills. *Continual fine-tuning* methods [Kirkpatrick et al., 2017, De Lange et al., 2021] update model weights on new data, but they typically require hand-curated datasets and are susceptible to catastrophic forgetting when data is noisy or unbalanced. Neither paradigm addresses the upstream question: *what should a model learn from?*

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

In biological intelligence, this problem is solved elegantly. The mammalian hippocampus does not store all sensory experience; instead, it selectively encodes salient events and, during sleep, replays and consolidates them into long-term neocortical memory [McClelland et al., 1995, Kumaran et al., 2016]. This process of *selective memory consolidation* ensures that only high-value experiences—those that are novel, surprising, or corrective—are used to update long-term knowledge.

Inspired by this principle, we propose **HAT (Hippocampus-Augmented Transformer)**, a multi-agent framework that equips a lightweight language model with the ability to *learn what to learn* from its own interactions. HAT introduces a dedicated *Hippocampus Agent* that monitors the base model's (the *Cortex*) interactions with users, abstracts raw dialogues into structured memory traces, selects high-value experiences using uncertainty, feedback, and novelty signals, and constructs training-ready replay data. During offline phases—analogous to *slow-wave sleep* (SWS) in the brain—curated experiences are replayed to fine-tune the Cortex, producing an improved model that re-enters the interaction loop. When the Cortex is uncertain or produces errors, HAT can also consult an external *Oracle* (a stronger teacher model), enabling a form of on-demand knowledge distillation.

Contributions. Our contributions are three-fold:

1. We formalise the problem of *selective experience curation* for self-improving language models and propose HAT, a memory-centric framework that decouples experience acquisition from parameter optimisation.
2. We design a Hippocampus Agent with three composable stages—*abstraction*, *selection*, and *replay construction*—that convert raw interactions into high-quality training signals using uncertainty estimation, user feedback, and novelty detection.
3. We conduct extensive experiments on knowledge-intensive QA, instruction following, and personalisation benchmarks, demonstrating that HAT achieves consistent improvements in sample efficiency, error-rate reduction, and long-horizon adaptation over strong fine-tuning and retrieval-augmented baselines.

2 Related Work

2.1 Continual Learning for Language Models

Continual learning (CL) aims to update model parameters on a stream of tasks without catastrophic forgetting [Kirkpatrick et al., 2017, Zenke et al., 2017]. Regularisation-based methods penalise changes to important weights [Kirkpatrick et al., 2017, Li and Hoiem, 2017], while replay-based methods store or generate past examples for interleaved training [De Lange et al., 2021, Scialom et al., 2022]. Recent work has extended CL to large language models, studying instruction tuning across sequential tasks [Wu et al., 2024] and domain-incremental adaptation [Ke et al., 2023]. However, existing CL methods assume a *given* data stream and do not address the problem of *which* experiences should be retained. HAT complements CL by introducing a principled experience selection mechanism upstream of parameter updates.

2.2 Memory-Augmented Neural Networks

Memory-augmented architectures equip neural networks with external memory stores that can be read from and written to [Graves et al., 2014, Sukhbaatar et al., 2015]. More recent work combines retrieval mechanisms with language models [Borgeaud et al., 2022, Zhong et al., 2022], enabling access to a large knowledge base at inference time. While these approaches augment *inference*, they do not leverage memory for *learning*: retrieved information is used contextually but does not update the model's parameters. In contrast, HAT uses its memory store (the Neocortex) as a curated training set that feeds back into parameter optimisation.

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) methods prepend retrieved passages to the model's input context [Lewis et al., 2020, Guu et al., 2020, Izacard et al., 2023]. RAG is effective for knowledge-intensive tasks but inherits the limitations of fixed-context approaches: the model cannot *learn* from retrieved content, context windows are bounded, and retrieval quality degrades when the knowledge base is noisy. HAT can be seen as going beyond RAG by closing the loop from retrieval to training: high-value interactions—including oracle-provided corrections—are not only used at inference time but are consolidated into the model's parameters.

2.4 Knowledge Distillation and Self-Improvement

Knowledge distillation transfers knowledge from a teacher model to a student [Hinton et al., 2015, Sanh et al., 2019]. Recent work explores *self-improving* language models that generate their own training data [Huang et al., 2023, Yuan et al., 2024, Chen et al., 2024] or use reinforcement learning from human/AI feedback [Ouyang et al., 2022, Bai et al., 2022]. These methods typically apply distillation or self-play uniformly, without selecting *which* experiences are most informative. HAT's Oracle consultation mechanism is a form of selective, on-demand distillation: the teacher is queried only when the student model is uncertain or has been corrected by the user, ensuring efficient use of the oracle's capacity.

2.5 Complementary Learning Systems Theory

Complementary learning systems (CLS) theory posits that the brain uses two interacting memory systems: the hippocampus for rapid encoding of specific episodes and the neocortex for slow extraction of statistical regularities [McClelland et al., 1995, Kumaran et al., 2016]. Sleep replay, particularly during slow-wave sleep, is hypothesised to mediate the transfer from hippocampus to neocortex [Rasch and Born, 2013, Walker, 2017]. Several works in machine learning have drawn on CLS for continual learning [Shin et al., 2017, van de Ven et al., 2020, Arani et al., 2022], but they focus on image classification and do not address the unique challenges of language model self-improvement from interactive experience. HAT directly operationalises the CLS framework for LLMs, with an explicit hippocampal selection stage, a neocortical memory store, and an offline SWS training phase.

3 HAT: Hippocampus-Augmented Transformer

We present **Hippocampus-Augmented Transformer (HAT)**, a memory-centric continual learning approach that enables language models to learn *what to learn* from interaction. HAT is inspired by hippocampal–neocortical memory consolidation in the mammalian brain, where salient experiences are rapidly encoded, selectively retained, and later replayed for gradual integration into long-term cortical memory. Rather than updating a model from all observed interactions, HAT separates online experience acquisition from offline model adaptation: during the *wake phase*, interaction traces are selectively consolidated into a long-term memory; during the *slow-wave sleep* (SWS) phase, selected traces are replayed as training signals to update the model.

As shown in Figure 1, HAT consists of four functional components. The **Cortex** $\mathcal{M}_\theta$ is a lightweight language model that interacts with users. The **Hippocampus Agent** $\mathcal{H}$ abstracts interactions into structured memory traces, estimates their learning value, and determines whether they should be retained. The **Neocortex** $\mathcal{N}$ is a persistent long-term memory that stores selected traces and their replay-ready training forms. The **SWS Trainer** periodically updates the Cortex by replaying curated memories from the Neocortex. An external **Oracle** $\mathcal{O}$ may be queried on demand when the Cortex is uncertain or when user feedback indicates an error.

3.1 Selective Learning Formulation

HAT treats continual adaptation as a selective learning problem over an interaction stream. Let

$$\mathcal{D}^{(t)} = \{x_i\}_{i=1}^{T_t} \tag{1}$$

denote the set of interactions observed during wake cycle t, where each interaction is represented as

$$x_i = (c_i, x_i, y_i, f_i), \tag{2}$$

with context c_i, user query x_i, model response y_i, and optional feedback signal f_i. The goal is not to learn from every interaction, but to identify a subset of experiences that are likely to improve the model under a limited adaptation budget.

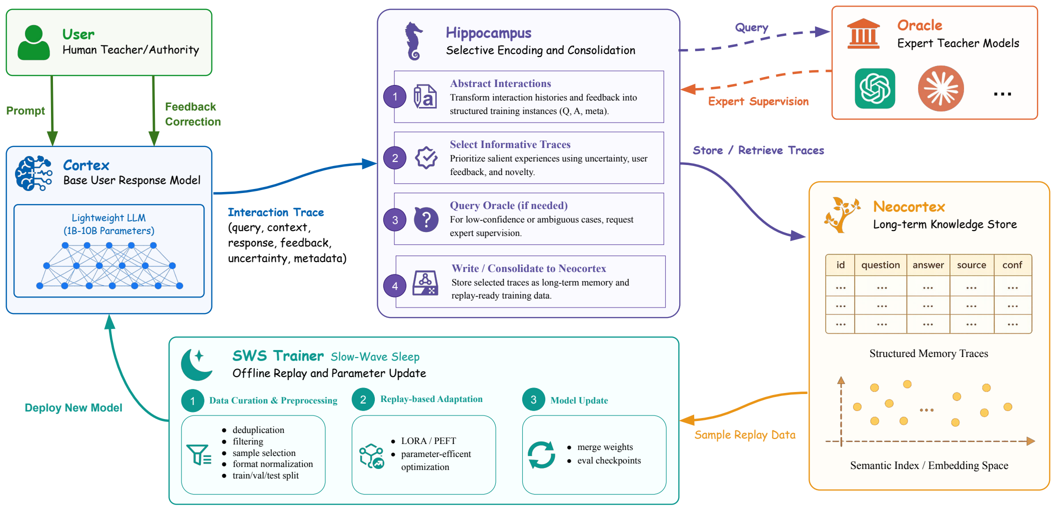

Figure 1: Overview of HAT, a hippocampus-inspired continual learning mechanism that converts selected interaction experiences into reusable training signals. During wake-phase interaction, the Cortex responds to user prompts while the Hippocampus encodes, selects, and optionally augments informative traces with Oracle supervision. The selected traces are consolidated into the Neocortex and replayed during slow-wave sleep (SWS), where the SWS Trainer updates the Cortex through parameter-efficient fine-tuning for continual self-improvement.

Ideally, one would select experiences according to their expected learning gain:

$$\mathcal{D}^{*(t)} = \arg \max_{\mathcal{D}' \subseteq \mathcal{D}^{(t)}} \mathbb{E}_{x \sim \mathcal{D}'} \left[\Delta \mathcal{L}(\theta; x) \right], \tag{3}$$

where $\Delta\mathcal{L}(\theta; x)$ denotes the expected reduction in future loss after adapting the model on experience x. Since this quantity is generally unobservable at selection time, HAT approximates expected learning gain using observable proxies: model uncertainty, external feedback, and semantic novelty. This converts continual learning from indiscriminate data accumulation into a selective memory consolidation process.

3.2 Wake–Sleep Learning Cycle

HAT alternates between two complementary phases.

Wake phase. During the wake phase, the Cortex $\mathcal{M}_\theta$ receives a user query x under context c and generates a response y. The resulting interaction is passed to the Hippocampus Agent, which compresses it into a memory trace, evaluates its utility, and determines whether it should be consolidated into the Neocortex. If the Cortex is uncertain or the user provides corrective feedback, the Hippocampus Agent may consult the Oracle to obtain additional supervision.

Sleep phase. During the SWS phase, selected traces stored in the Neocortex are replayed as training examples. The SWS Trainer updates the Cortex parameters using a curated replay distribution:

$$\theta^{(t+1)} = \mathrm{SWS}\left(\theta^{(t)}, \mathcal{N}^{(t)}\right). \tag{4}$$

The updated model then becomes the Cortex for the next wake cycle. Thus, HAT separates the decision of *what should be learned* from the optimization process that performs the actual learning.

3.3 Cortex: Online Interaction Model

The Cortex is an autoregressive Transformer language model $\mathcal{M}_\theta$ with parameters θ. It can be instantiated by any lightweight model suitable for deployment, such as a model in the 0.5–10B parameter range. Given context c and query x, the Cortex generates a response $y = (y_1, \ldots, y_{|y|})$ according to

$$p_\theta(y \mid c, x) = \prod_{j=1}^{|y|} p_\theta\left(y_j \mid c, x, y_{<j}\right). \tag{5}$$

In addition to producing a response, the Cortex provides an uncertainty estimate $U(x)$ for each interaction. In practice, $U(x)$ can be computed from predictive entropy, token-level confidence, self-consistency variance, or disagreement between sampled generations. A simple entropy-based estimate is

$$U(x) = \frac{1}{|y|} \sum_{j=1}^{|y|} H\left(p_\theta(\cdot \mid c, x, y_{<j})\right), \tag{6}$$

where $H(\cdot)$ denotes entropy. High uncertainty indicates a potential epistemic gap and serves as one signal for selective memory consolidation.

3.4 Hippocampus Agent: Selective Memory Consolidation

The Hippocampus Agent is the central mechanism of HAT. It transforms raw interactions into structured memory traces and determines which traces should be consolidated into the Neocortex. This process consists of three operations: abstraction, selection, and replay construction.

3.4.1 Experience Abstraction

Raw interaction logs are often verbose, redundant, and unsuitable for direct training. The abstraction operation maps an interaction $x = (c, x, y, f)$ into a compact memory trace:

$$m = \mathcal{H}_{\text{abs}}(c, x, y, f). \tag{7}$$

The trace m preserves the learning-relevant content of the episode, including the user intent, the model response, the corrected answer if available, and relevant contextual constraints. This operation is analogous to rapid hippocampal encoding, where an experience is compressed into a retrievable trace rather than stored as an unfiltered sensory stream.

In implementation, $\mathcal{H}_{\text{abs}}$ can be instantiated as a prompting procedure, a small summarization model, or the Cortex itself under an instruction template. A memory trace may be represented as

$$m = \left(q,\ a_{\text{cortex}},\ a_{\text{target}},\ r,\ \xi\right), \tag{8}$$

where q is the normalized query, a_{cortex} is the Cortex response, a_{target} is the preferred target answer, r is the rationale or correction signal, and ξ contains timestamp, source, uncertainty, feedback, and novelty statistics.

3.4.2 Selection via Write Policy

Not all traces should be stored or replayed. HAT therefore defines a write policy π_{write} that estimates the consolidation value of each memory trace. Given a trace m, the policy computes

$$\text{score}(m) = \alpha U(m) + \beta F(m) + \gamma N(m), \tag{9}$$

where U, F, and N denote uncertainty, feedback, and novelty, respectively.

Uncertainty. $U(m)$ estimates the extent to which the Cortex is unreliable on the corresponding interaction. High uncertainty suggests that the trace lies near the boundary of the model's current competence and may yield high learning value.

Feedback. $F(m)$ captures explicit or implicit supervision. For example, user corrections, negative ratings, or contradiction signals can be mapped to a binary or graded value. When an explicit correction is provided, $F(m)$ is assigned high priority to ensure that verified errors are consolidated.

Novelty. $N(m)$ measures the semantic distance between the current trace and the existing Neocortex memory. Let $e(m)$ be an embedding of the trace and $\mathcal{N}$ be the current memory store. A practical novelty score is

$$N(m) = 1 - \max_{m' \in \mathcal{N}} \cos\left(e(m), e(m')\right). \tag{10}$$

Novel traces are prioritized because they expand coverage of underrepresented regions in the model's knowledge space.

The final write decision is

$$z(m) = \mathbb{1}\left[\mathrm{score}(m) > \tau\right], \tag{11}$$

where τ is a tunable consolidation threshold. The coefficients α, β, γ control the relative importance of epistemic uncertainty, external supervision, and semantic coverage.

This write policy can be interpreted as a tractable surrogate for Equation 3: uncertainty approximates knowledge gaps, feedback identifies externally verified errors, and novelty encourages memory diversity.

3.4.3 Replay Construction

Selected memory traces must be converted into training-ready signals. For each retained trace m, the Replay Builder produces one or more supervised examples:

$$(x^{\mathrm{train}}, y^{\mathrm{train}}) \sim p_{\mathrm{replay}}(\cdot \mid m) = \mathcal{H}_{\mathrm{replay}}(m). \tag{12}$$

The replay distribution can include direct instruction–response pairs, corrected QA pairs, rationale-augmented examples, or contrastive pairs that distinguish the original erroneous response from the preferred target.

This step is important because memory consolidation in HAT is not merely storage. A retained trace becomes useful only when it is transformed into a form that can drive gradient-based learning. Thus, the Hippocampus Agent converts interaction experience into reusable training signals.

3.5 Oracle: On-Demand External Supervision

The Oracle $\mathcal{O}$ is an external teacher used only when additional supervision is expected to be valuable. Rather than distilling a large teacher model over an entire corpus, HAT queries the Oracle selectively. The Oracle is invoked when the uncertainty exceeds a threshold $\tau_{\mathcal{O}}$ or when user feedback indicates an error:

$$y^{\mathcal{O}} = \mathcal{O}(c, x), \quad \text{if } U(x) > \tau_{\mathcal{O}} \text{ or } F(x) > 0. \tag{13}$$

The Oracle response may replace, refine, or supplement the Cortex response in the memory trace:

$$a_{\mathrm{target}} = \mathrm{Merge}\left(a_{\mathrm{cortex}}, a_{\mathrm{user}}, a_{\mathcal{O}}\right), \tag{14}$$

where a_{user} denotes a user-provided correction when available. This mechanism implements budget-aware supervision: teacher knowledge is acquired primarily at the frontier of the student's uncertainty or error, where it is most likely to improve future behavior.

3.6 Neocortex: Long-Term Memory Store

The Neocortex $\mathcal{N}$ is a persistent memory store containing selected traces and their replay-ready training forms. At wake cycle t, the memory is updated as

$$\mathcal{N}^{(t)} = \mathcal{N}^{(t-1)} \cup \{m_i \;:\; z(m_i) = 1\}. \tag{15}$$

Each memory entry stores both semantic content and training metadata:

$$\mathcal{N} = \{(m_i,\ \mathcal{R}_i,\ \mathrm{score}_i,\ \xi_i)\}_{i=1}^{|\mathcal{N}|}, \tag{16}$$

where $\mathcal{R}_i$ denotes replay examples generated from trace m_i. In implementation, the Neocortex can be realized as a vector-indexed datastore with structured fields, allowing retrieval, deduplication, pruning, and replay sampling.

To keep memory bounded, HAT may prune low-value or redundant traces:

$$\mathcal{N} \leftarrow \mathrm{TopK}\left(\mathcal{N},\ \mathrm{score}(m)\right), \tag{17}$$

or apply diversity-aware sampling to avoid overrepresenting frequent interaction patterns. This prevents indiscriminate accumulation and maintains the Neocortex as a curated long-term learning substrate.

3.7 Slow-Wave Sleep Trainer

During the SWS phase, HAT updates the Cortex by replaying selected experiences from the Neocortex. Let $\mathcal{B} \subseteq \mathcal{N}$ denote a replay batch sampled from the memory store. The SWS Trainer performs empirical risk minimization over the curated replay distribution:

$$\min_{\theta}\ \mathbb{E}_{(x,y)\sim p_{\mathrm{replay}}}\left[\mathcal{L}_{\mathrm{sup}}(\theta; x, y)\right] + \mathcal{R}(\theta). \tag{18}$$

In practice, the supervised loss is the token-level negative log-likelihood:

$$\mathcal{L}_{\mathrm{sup}} = -\frac{1}{|\mathcal{B}|} \sum_{(x,y)\in\mathcal{B}} \sum_{j=1}^{|y|} \log p_{\theta}\left(y_j \mid x, y_{<j}\right). \tag{19}$$

For traces involving Oracle supervision, HAT optionally includes a distillation term:

$$\mathcal{L}_{\mathrm{KD}} = \frac{1}{|\mathcal{B}_{\mathcal{O}}|} \sum_{x\in\mathcal{B}_{\mathcal{O}}} \mathrm{KL}\left(p_{\mathcal{O}}(\cdot \mid x) \,\|\, p_{\theta}(\cdot \mid x)\right), \tag{20}$$

where $\mathcal{B}_{\mathcal{O}}$ is the subset of replay examples that contain Oracle-derived targets.

To reduce catastrophic forgetting, the SWS objective may include a stability regularizer:

$$\mathcal{L}_{\mathrm{stab}} = \frac{\mu}{2} \sum_i F_i \left(\theta_i - \theta_i^{\mathrm{old}}\right)^2, \tag{21}$$

where F_i denotes the estimated importance of parameter θ_i, such as the diagonal Fisher information as in elastic weight consolidation [Kirkpatrick et al., 2017].

The full SWS objective is

$$\mathcal{L}_{\mathrm{SWS}} = \mathcal{L}_{\mathrm{sup}} + \lambda_{\mathrm{KD}}\mathcal{L}_{\mathrm{KD}} + \lambda_{\mathrm{stab}}\mathcal{L}_{\mathrm{stab}}. \tag{22}$$

The Cortex parameters are then updated as

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}_{\mathrm{SWS}}\left(\theta^{(t)}; \mathcal{B}^{(t)}\right). \tag{23}$$

For efficient deployment, HAT can instantiate this update using parameter-efficient fine-tuning methods such as LoRA or adapters, while keeping most model parameters frozen. This makes the SWS phase practical for small or edge-deployed language models.

3.8 Iterative Self-Improvement

The complete HAT process forms a recurrent wake–sleep learning loop. At cycle t, the current Cortex interacts with users and produces an interaction stream $\mathcal{D}^{(t)}$. The Hippocampus Agent abstracts and filters this stream into selected memory traces. The Neocortex stores these traces as long-term memory. The SWS Trainer then replays the selected traces to produce the next Cortex:

$$\theta^{(t+1)} = \mathcal{A}\left(\theta^{(t)}, \pi_{\mathrm{write}}\left(\mathcal{D}^{(t)}, \mathcal{N}^{(t)}\right)\right), \tag{24}$$

where $\mathcal{A}$ denotes the SWS adaptation operator and π_{write} denotes the selective consolidation policy.

This loop creates a self-improving process in which each model generation produces new interaction evidence, the Hippocampus Agent decides which experiences are worth retaining, and the SWS phase converts retained experiences into parameter updates. Importantly, HAT does not require the model to store all interactions or continuously update online. Instead, it learns through selective consolidation: wake-phase interactions determine *what* should be learned, while sleep-phase replay determines *how* the model is updated.

Algorithm 1 summarizes the full procedure.

Algorithm 1: HAT wake–sleep learning loop

Input: Initial Cortex parameters $\theta^{(0)}$, Neocortex memory $\mathcal{N}^{(0)}$, Oracle $\mathcal{O}$, thresholds $\tau, \tau_{\mathcal{O}}$

for *wake–sleep cycle* $t = 0, 1, 2, \ldots$ **do**
  Collect wake-phase interactions $\mathcal{D}^{(t)}$
  for *each interaction* $x = (c, x, y, f) \in \mathcal{D}^{(t)}$ **do**
    Estimate uncertainty $U(x)$ from Cortex $\mathcal{M}_\theta$
    Encode memory trace $m \leftarrow \mathcal{H}_{\text{abs}}(x)$
    if $U(x) > \tau_{\mathcal{O}}$ *or* $F(x) > 0$ **then**
      Query Oracle $y^{\mathcal{O}} \leftarrow \mathcal{O}(c, x)$
      Update trace target using Oracle/user supervision
    Compute novelty $N(m)$ against $\mathcal{N}^{(t)}$
    Compute $\text{score}(m) = \alpha U + \beta F + \gamma N$
    if $\text{score}(m) > \tau$ **then**
      Build replay examples $\mathcal{R} \leftarrow \mathcal{H}_{\text{replay}}(m)$
      Store $(m, \mathcal{R}, \text{score}, \xi)$ in $\mathcal{N}^{(t)}$
  Sample replay batch $\mathcal{B}^{(t)} \subseteq \mathcal{N}^{(t)}$
  Update Cortex via SWS training: $\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \nabla_\theta \mathcal{L}_{\text{SWS}}$
  Optionally prune redundant or low-value memories from $\mathcal{N}^{(t)}$

4 Experiments

4.1 Benchmarks and Tasks

We evaluate HAT on three families of tasks that probe different aspects of self-improvement:

Knowledge-intensive question answering. We use Natural Questions (NQ) [Kwiatkowski et al., 2019] and TriviaQA [Joshi et al., 2017] in a *streaming* setup: questions arrive sequentially and the model must answer without access to a retrieval corpus. After each answer, the model may receive a binary correctness signal or an explicit user correction.

Instruction following. We adopt a subset of the Alpaca-Eval benchmark [Li et al., 2023] and MT-Bench [Zheng et al., 2023], where instructions are presented in temporal batches that gradually increase in complexity. This tests HAT's ability to internalise and generalise from corrective feedback on instruction-following quality.

Personalisation. We construct a personalisation benchmark based on LaMP [Salemi et al., 2024], where a simulated user provides preferences and corrections over successive interactions. The model is evaluated on how well it adapts to individual user style and factual preferences over time.

4.2 Baselines

We compare HAT against the following approaches:

- **Frozen**: the base Cortex model without any adaptation.
- **RAG** [Lewis et al., 2020]: retrieval-augmented generation using a dense retriever over the interaction history.
- **Naïve Replay**: continual fine-tuning on all interaction logs without selection.
- **EWC Replay** [Kirkpatrick et al., 2017]: replay with elastic weight consolidation but without experience curation.
- **DER++** [Buzzega et al., 2020]: dark experience replay with knowledge distillation, a strong continual learning baseline.
- **Self-Play** [Chen et al., 2024]: the model generates and filters its own training data via self-consistency.

Table 1: Main results after 10 SWS cycles. We report accuracy (%) on knowledge QA (NQ / TriviaQA, exact match), instruction following (MT-Bench, GPT-4 judge score $\times 10$), and personalisation (LaMP, ROUGE-L $\times 100$). Best in **bold**; second best underlined. [†]Full Distillation uses the oracle for *every* query (upper bound).

	Knowledge QA			Instruction Following			Personalisation		
Method	1.5B	3.8B	8B	1.5B	3.8B	8B	1.5B	3.8B	8B
Frozen	–	–	–	–	–	–	–	–	–
RAG	–	–	–	–	–	–	–	–	–
Naïve Replay	–	–	–	–	–	–	–	–	–
EWC Replay	–	–	–	–	–	–	–	–	–
DER++	–	–	–	–	–	–	–	–	–
Self-Play	–	–	–	–	–	–	–	–	–
Full Distill.[†]	–	–	–	–	–	–	–	–	–
HAT (ours)	–	–	–	–	–	–	–	–	–

- **Full Distillation**: fine-tuning the student on the oracle's responses for *all* queries (upper bound on oracle usage).

4.3 Implementation Details

Base models. We instantiate the Cortex with three model families at different scales: Qwen-2.5-1.5B, Phi-3-mini-3.8B, and LLaMA-3-8B. All models are instruction-tuned variants.

Oracle. We use GPT-4o as the default oracle $\mathcal{O}$. We also report results with a smaller oracle (LLaMA-3-70B) to study oracle quality sensitivity.

Hippocampus Agent. The abstraction and replay-builder modules reuse the Cortex itself (with dedicated prompts), making HAT lightweight and self-contained. The selection scoring uses the Cortex's own predictive entropy for U, cosine similarity in the model's embedding space for N, and binary user-correction flags for F.

Hyperparameters. We set $\alpha = 0.4$, $\beta = 0.4$, $\gamma = 0.2$ in Eq. (9) based on validation-set tuning. The write threshold is $\tau = 0.3$. Oracle consultation is triggered when $U > \tau_{\mathcal{O}} = 0.7$ or when $F = 1$. SWS training uses LoRA (rank 16) with AdamW ($\eta = 2 \times 10^{-5}$), batch size 32, and 3 epochs per cycle. The EWC coefficient is $\mu = 0.1$. A SWS cycle is triggered every 500 interactions.

Evaluation protocol. We report metrics averaged over 3 random seeds. Each experiment simulates 5,000 sequential interactions (10 SWS cycles). We measure: (1) **accuracy** (exact match or ROUGE-L depending on task), (2) **error rate** (fraction of incorrect responses), (3) **sample efficiency** (accuracy gain per training example), and (4) **forgetting rate** (performance drop on earlier tasks after adaptation).

5 Results

5.1 Main Results

Table 1 summarises the performance of HAT and all baselines after 5,000 interactions (10 SWS cycles) across three benchmark families and three base model scales.

HAT consistently outperforms non-parametric baselines.

HAT surpasses continual learning baselines.

Selective curation closes the gap to full distillation.

[Sample Efficiency Plot Placeholder]

[Error Rate Over Time Placeholder]

Figure 2: **Left:** Accuracy vs. number of training examples consumed (sample efficiency). **Right:** Error rate over successive SWS cycles. HAT achieves higher accuracy with fewer examples and reduces errors faster than baselines.

Table 2: Impact of oracle budget on HAT performance (Phi-3-3.8B, NQ benchmark). "% Oracle" is the fraction of interactions that trigger an oracle call.

Setting	**% Oracle**	**Accuracy**	**Error Rate**
HAT (no oracle)	0%	–	–
HAT ($\tau_{\mathcal{O}} = 0.9$)	–%	–	–
HAT ($\tau_{\mathcal{O}} = 0.7$)	–%	–	–
HAT ($\tau_{\mathcal{O}} = 0.5$)	–%	–	–
Full Distillation	100%	–	–

5.2 Sample Efficiency

Figure 2 plots accuracy as a function of the number of training examples consumed.

5.3 Oracle Budget Analysis

A practical concern is the cost of oracle consultations. Table 2 reports performance and the number of oracle calls made under different oracle-budget constraints.

6 Analysis

6.1 Ablation Study

To understand the contribution of each component, we ablate key modules of HAT using the Phi-3-3.8B Cortex on the NQ benchmark. Results are summarised in Table 3.

Abstraction is critical.

Selection outperforms random curation.

All three selection signals contribute.

EWC prevents forgetting.

6.2 Effect of SWS Cycle Frequency

We vary the number of interactions between SWS cycles ($N \in \{100, 250, 500, 1000, 2000\}$) and measure final accuracy and computational cost.

6.3 Qualitative Analysis

What does the Hippocampus select? We analyse the distribution of memory traces written to the Neocortex across interaction types.

Table 3: Ablation study on the NQ benchmark (Phi-3-3.8B). Each row removes or replaces one component. Δ indicates the accuracy change relative to the full HAT system.

Variant	**Accuracy**	$\boldsymbol{\Delta}$
HAT (full)	–	—
w/o Abstraction	–	–
w/o Selection (random write)	–	–
w/o Novelty signal ($\gamma = 0$)	–	–
w/o Uncertainty signal ($\alpha = 0$)	–	–
w/o Feedback signal ($\beta = 0$)	–	–
w/o Oracle consultation	–	–
w/o EWC regularisation	–	–
w/o Replay Builder (raw traces)	–	–

[SWS Frequency Plot Placeholder]

[Forgetting Rate Plot Placeholder]

Figure 3: **Left:** Final accuracy as a function of SWS cycle frequency. **Right:** Forgetting rate across SWS cycles for HAT vs. baselines.

Memory evolution over time. We visualise the Neocortex content across SWS cycles using t-SNE embeddings.

Case study: error correction propagation. We trace a specific user correction through the HAT pipeline—from raw interaction to abstracted trace, selection, replay construction, and its effect on model behaviour after SWS training.

6.4 Scaling Behaviour

We examine how HAT's gains vary with Cortex model size (1.5B, 3.8B, 8B).

7 Conclusion

We introduced **HAT (Hippocampus-Augmented Transformer)**, a memory-centric framework that enables lightweight language models to self-improve by learning *what to learn* from interactive experience. Drawing on complementary learning systems theory, HAT decouples experience acquisition (online) from parameter optimisation (offline) through three key mechanisms: abstraction of raw interactions into structured memory traces, selective curation based on uncertainty, feedback, and novelty, and offline replay-driven fine-tuning during slow-wave sleep phases. An on-demand oracle consultation mechanism provides efficient, targeted knowledge distillation only where the model is most uncertain.

Experiments across knowledge-intensive QA, instruction following, and personalisation benchmarks demonstrate that HAT consistently outperforms strong fine-tuning and retrieval-augmented baselines in accuracy, sample efficiency, and resistance to catastrophic forgetting, while using a fraction of the oracle budget required by full distillation.

Limitations and future work. Several directions remain open. First, the current selection heuristic (Eq. 9) uses a linear combination of fixed signals; learning the write policy end-to-end via meta-learning is a promising extension. Second, our evaluation simulates user interactions from existing benchmarks; deploying HAT with real users in a longitudinal study would provide stronger evidence of its practical impact. Third, while HAT targets lightweight models (0.5–10 B), scaling the framework to larger models and investigating the interplay between model capacity and memory curation is of both theoretical and practical interest. Finally, privacy considerations arise when storing user interactions in the Neocortex; integrating differential privacy or federated learning into the SWS training phase is an important direction for responsible deployment.

References

Elahe Arani, Fahad Sarfraz, and Bahram Zonooz. Learning fast, learning slow: A general continual learning method based on complementary learning system. In *International Conference on Learning Representations*, 2022.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. *arXiv preprint arXiv:2112.04426*, 2022.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: A strong, simple baseline. In *Advances in Neural Information Processing Systems*, volume 33, pages 15920–15930, 2020.

Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024.

Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2021.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. In *arXiv preprint arXiv:1410.5401*, 2014.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. REALM: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning*, pages 3929–3938, 2020.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. *arXiv preprint arXiv:2210.11610*, 2023.

Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43, 2023.

Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611, 2017.

Zixuan Ke, Yijia Shao, Haowei Lin, Tatsuya Konishi, Gyuhak Kim, and Bing Liu. Continual pre-training of language models. *arXiv preprint arXiv:2302.03241*, 2023.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences*, volume 114, pages 3521–3526, 2017.

Dharshan Kumaran, Demis Hassabis, and James L McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20 (7):512–534, 2016.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Alpacaeval: An automatic evaluator of instruction-following models. *GitHub repository*, 2023.

Zhizhong Li and Derek Hoiem. Learning without forgetting. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 40, pages 2935–2947, 2017.

James L McClelland, Bruce L McNaughton, and Randall C O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

Björn Rasch and Jan Born. About sleep's role in memory. *Physiological Reviews*, 93(2):681–766, 2013.

Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. LaMP: When large language models meet personalization. *arXiv preprint arXiv:2304.11406*, 2024.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. In *NeurIPS EMC^2 Workshop*, 2019.

Thomas Scialom, Tuhin Chakrabarty, and Smaranda Muresan. Fine-tuned language models are continual learners. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6fenêtre–6102, 2022.

Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Gido M van de Ven, Hava T Siegelmann, and Andreas S Tolias. Brain-inspired replay for continual learning with artificial neural networks. *Nature Communications*, 11(1):4069, 2020.

Matthew Walker. *Why We Sleep: Unlocking the Power of Sleep and Dreams*. Scribner, 2017.

Tongtong Wu, Linhao Luo, Yuan-Fang Li, Shirui Pan, Thuy-Trang Vu, and Gholamreza Haffari. Continual learning for large language models: A survey. *arXiv preprint arXiv:2402.01364*, 2024.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, pages 3987–3995, 2017.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.

Zexuan Zhong, Tao Lei, and Danqi Chen. Training language models with memory augmentation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5657–5673, 2022.

A Algorithm Pseudocode

Algorithm 2: HAT — Hippocampus-Augmented Transformer Self-Improvement Loop

Input : Base model $\mathcal{M}_\theta$ (Cortex), Hippocampus Agent $\mathcal{H}$, Oracle $\mathcal{O}$, empty Neocortex $\mathcal{N} \leftarrow \emptyset$
Input : SWS interval N, write threshold τ, oracle threshold $\tau_{\mathcal{O}}$, selection weights α, β, γ

$t \leftarrow 0$
for *each user interaction* (c, x) **do**
  $y \leftarrow \mathcal{M}_\theta(c, x)$ `// Cortex generates response`
  $U \leftarrow \mathrm{H}\big(p_\theta(y \mid c, x)\big)$ `// Estimate uncertainty`
  Observe user feedback F (correction signal, if any)
  `// -- Hippocampus Agent --`
  $m \leftarrow \mathcal{H}_{\mathrm{abs}}(c, x, y)$ `// Abstraction`
  if $U > \tau_{\mathcal{O}}$ ***or*** $F = 1$ **then**
    $y^{\mathcal{O}} \leftarrow \mathcal{O}(c, x)$ `// Consult Oracle`
    Update m with oracle response $y^{\mathcal{O}}$
  Compute novelty: $N \leftarrow \mathrm{NoveltyScore}(m, \mathcal{N})$
  Compute selection score: $s \leftarrow \alpha\, U + \beta\, F + \gamma\, N$
  if $s > \tau$ **then**
    $(x^{\mathrm{train}}, y^{\mathrm{train}}) \leftarrow \mathcal{H}_{\mathrm{replay}}(m)$ `// Replay Builder`
    $\mathcal{N} \leftarrow \mathcal{N} \cup \{(x^{\mathrm{train}}, y^{\mathrm{train}}, s)\}$ `// Write to Neocortex`
  $t \leftarrow t + 1$
  `// -- Slow-Wave Sleep --`
  if $t \bmod N = 0$ **then**
    Sample replay batch $\mathcal{B} \sim \mathcal{N}$ (priority sampling by s)
    $\theta \leftarrow \theta - \eta\, \nabla_\theta [\mathcal{L}_{\mathrm{SWS}}(\theta; \mathcal{B}) + \mathcal{L}_{\mathrm{reg}}(\theta)]$ `// SWS update`
    Update Fisher information F for EWC
return Updated model $\mathcal{M}_\theta$

Table 4: Hyperparameter sensitivity analysis on the NQ benchmark (Phi-3-3.8B).

Hyperparameter	Range Tested	Best Value	Sensitivity
Write threshold τ	$\{0.1, 0.2, 0.3, 0.5, 0.7\}$	0.3	Medium
Oracle threshold $\tau_{\mathcal{O}}$	$\{0.5, 0.6, 0.7, 0.8, 0.9\}$	0.7	Low
Uncertainty weight α	$\{0.2, 0.3, 0.4, 0.5, 0.6\}$	0.4	Medium
Feedback weight β	$\{0.2, 0.3, 0.4, 0.5, 0.6\}$	0.4	Low
Novelty weight γ	$\{0.1, 0.2, 0.3, 0.4\}$	0.2	Low
LoRA rank	$\{4, 8, 16, 32\}$	16	Low
SWS learning rate η	$\{1, 2, 5\} \times 10^{-5}$	2×10^{-5}	Medium
EWC coefficient μ	$\{0.01, 0.05, 0.1, 0.5, 1.0\}$	0.1	Medium
SWS interval N	$\{100, 250, 500, 1000, 2000\}$	500	Low

Table 5: Computational overhead of HAT components (Phi-3-3.8B on a single NVIDIA A100 GPU).

Component	Time per Interaction	Time per SWS Cycle
Cortex inference	– ms	—
Hippocampus (abstraction)	– ms	—
Hippocampus (selection)	– ms	—
Oracle consultation	– ms	—
SWS training (3 epochs)	—	– min
Total overhead	– ms	– min

B Hyperparameter Sensitivity

C Additional Experimental Results

D Prompts Used by the Hippocampus Agent

E Computational Cost Analysis

F Broader Impact Statement

HAT enables on-device language models to improve from user interactions, which raises considerations in both positive and negative directions.

Positive impacts. HAT democratises model adaptation by allowing lightweight models on personal devices to improve without requiring cloud-based retraining. This can reduce reliance on centralised AI services and give users more control over their AI assistants. The selective memory mechanism also implies that not all data is stored, which can be beneficial for privacy.

Potential risks. Storing user interactions, even in abstracted form, poses privacy risks. If the Neocortex memory is compromised, sensitive information could be exposed. Additionally, a model that learns from user corrections could amplify biases if the corrections themselves are biased. We recommend deploying HAT with differential privacy mechanisms and content filtering to mitigate these risks.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.